Project IV:

Beautiful Subsequence

Advanced Data Structures and Algorithms Analysis

Group1: Lin Zhiyu, Hu Junyu, Huang Zhenhua

College of Computer Science and Technology
Zhejiang University

November 17, 2025

Contents

1 Introduction **3**

2 Algorithm Specification **3**
2.1 Data Structures 3
2.2 Algorithm 1: Complementary Counting with Fenwick Tree 3
2.2.1 Main Idea 3
2.2.2 Dynamic Programming Formulation 3
2.2.3 Fenwick Tree Optimization 4
2.2.4 Pseudocode 4
2.3 Other Algorithms 5
2.4 Workflow 5

3 Test Cases and Results **5**
3.1 Test Environment 5
3.2 Test Case Design 6
3.3 Experimental Results 6

4 Performance Analysis **7**
4.1 Complexity Analysis of the Optimized Algorithm 7
4.2 Analysis of Experimental Results 7
4.3 Comparative Analysis and Key Observations 8

5 Conclusion **9**

6 Declaration **9**

1 Introduction

This project describes a combinatorial counting problem focused on identifying and counting "beautiful subsequences" within a given integer sequence. A subsequence is defined as beautiful if it contains at least one pair of adjacent elements whose absolute difference is no larger than a specified integer m. The challenge lies in efficiently counting these subsequences, especially given the constraints where the length of the sequence n can be as large as 10^5.

Beside the brute-force method of generating and checking all subsequences, which is infeasible for large n, a naive idea may be to directly count beautiful subsequences. However, this approach is complicated by the potential for overcounting, as a single subsequence may satisfy the "beautiful" condition in multiple ways.

To overcome these challenges, we adopt a complementary counting strategy. Instead of counting beautiful subsequences directly, we count their complement: "ugly" subsequences, where for every pair of adjacent elements, the absolute difference is strictly greater than m. This complementary problem can be efficiently solved using dynamic programming, with the aid of a Fenwick Tree (Binary Indexed Tree) to optimize state transitions.

2 Algorithm Specification

2.1 Data Structures

To implement the algorithm efficiently, we chose to use the following data structures:

- **Array:** We use a standard array to store the input integer sequence.
- **Fenwick Tree (Binary Indexed Tree):** This is the core data structure for our solution. A Fenwick Tree is a data structure that can efficiently update element values and calculate prefix sums in a table of numbers. In our algorithm, it is used to maintain the counts of "ugly" subsequences ending with specific values.

2.2 Algorithm 1: Complementary Counting with Fenwick Tree

This is our primary and most efficient algorithm. The core strategy is to count the complement of the desired set, which simplifies the counting logic significantly.

2.2.1 Main Idea

Instead of directly counting "beautiful" subsequences, which is complex due to overcounting, we count the number of "ugly" subsequences. An ugly subsequence is defined as one where for every pair of adjacent elements, the absolute difference is strictly greater than m. The final answer is then obtained by subtracting the count of ugly subsequences from the total number of possible subsequences (of length ≥ 2).

2.2.2 Dynamic Programming Formulation

We define a DP state, $dp[v]$, as the number of ugly subsequences ending with the value v. When we process a new element a_i from the input sequence, the number of new ugly

subsequences ending with a_i is given by:

$$dp[a_i] = 1 + \sum_{v \text{ s.t. } |a_i - v| > m} dp[v]$$

The '1' accounts for the subsequence consisting only of a_i itself. The summation term represents extending all existing ugly subsequences that can precede a_i without violating the "ugly" condition.

2.2.3 Fenwick Tree Optimization

A naive calculation of the summation term for each element would take $O(V)$ time, where V is the maximum possible value of an element, leading to an overall complexity of $O(N \cdot V)$. To optimize this, we observe that the condition $|a_i - v| > m$ is equivalent to $v < a_i - m$ or $v > a_i + m$. This transforms the problem into two range sum queries.

We use a Fenwick Tree (also known as a Binary Indexed Tree) to maintain the sums of $dp[v]$ values. This data structure allows us to perform both point updates and prefix sum queries in $O(\log V)$ time. For each a_i, the required sum can be calculated as:

$$\text{Sum} = \text{query}(a_i - m - 1) + (\text{query}(\text{MAX_VAL}) - \text{query}(a_i + m))$$

This optimization reduces the time complexity of each DP step to $O(\log V)$, making the total time complexity $O(N \log V)$.

2.2.4 Pseudocode

```
initialize FenwickTree fenwickTree
dp = array of size MAX_VAL initialized to 0
totalUgly = 0
for i from 1 to n do
    a_i = input[i]
    sumUgly = fenwickTree.query(a_i - m - 1) + (fenwickTree.query(MAX_VAL) - fenwickTree.query(a_i + m))
    dp[a_i] = (1 + sumUgly) mod MOD
    fenwickTree.update(a_i, dp[a_i])
    totalUgly = (totalUgly + dp[a_i]) mod MOD
end for
totalSubsequences = (2^n - n - 1) mod MOD
result = (totalSubsequences - totalUgly + MOD) mod MOD
```

Listing 1: Complementary Counting with Fenwick Tree

```
ElementType fenwickTree[MAX_VAL]
function fenwickTree.update(index, value)
    while index < MAX_VAL do
        fenwickTree[index] = (fenwickTree[index] + value) mod MOD
        index += index & (-index)
    end while
end function

function fenwickTree.query(index)
    result = 0
    while index > 0 do
        result = (result + fenwickTree[index]) mod MOD
        index -= index & (-index)
```

```
        end while
        return result
    end function
```

Listing 2: Fenwick Tree Implementation

2.3 Other Algorithms

In order to provide a comprehensive analysis, we also implemented and tested the following alternative algorithms:

- **Brute Force Enumeration:** This method generates all possible subsequences and checks each one for the "beautiful" condition. While straightforward, this approach is computationally infeasible for large n due to its exponential time complexity of $O(2^n)$.
- **Direct Counting with Dynamic Programming:** This approach attempts to count beautiful subsequences directly using a DP formulation. However, it suffers from overcounting issues and requires complex handling to avoid double counting, leading to a higher time complexity of $O(N^2)$.
- **Complementary Counting without Fenwick Tree:** This method uses a simple array to store DP states without the Fenwick Tree optimization. While it correctly implements the complementary counting strategy, it results in a time complexity of $O(N \cdot V)$, which is impractical for large input sizes.

2.4 Workflow

To measure the performance of each algorithm, we designed a testing-version for every algorithm, and a main driver program to execute them with randomized inputs. The workflow is as follows:

1. **Input Generation:** The `test_generator.c` module generates random test cases based on specified parameters, including the size of the input sequence n and the maximum value range.
2. **Algorithm Execution:** The main driver program sequentially runs each of the four algorithms on the generated input, measuring the execution time for each.
3. **Data Collection:** Execution times are recorded and stored for analysis.
4. **Result Processing:** The collected data is stored in a CSV file and visualized using Python's Matplotlib library to create performance comparison plots.

3 Test Cases and Results

3.1 Test Environment

The experiments were conducted on the following system:

- **Operating System:** Ubuntu 22.04 LTS

- **CPU:** 13th Gen Intel Core i7-13620H
- **Memory:** 16 GB RAM
- **Compiler:** GCC 11.4.0 with -O2 optimization

3.2 Test Case Design

To validate the correctness and efficiency of our algorithms, we designed a testing strategy focused on average-case performance under random inputs, which is implemented by our main driver program.

- **Randomized Inputs:** The primary method for testing involves generating random data. For each test run with a given size n, the `test_generator.c` module creates an array of n integers. Each integer is chosen uniformly at random from the range $[1, \text{MAX_VAL}]$. This approach allows us to assess the average-case performance of the algorithms across a wide spectrum of inputs.
- **Variable Input Size (N):** We test our algorithms across a range of input sizes to analyze how their runtime scales, choosing appropriate ranges for each algorithm based on its complexity.
- **Fixed Parameter (m):** In our tests, the parameter m is consistently set to a fixed value derived from the maximum value range: $m = (\text{MAX_VAL} + 1)/3$. This ensures that the comparison between different runs and algorithms is fair.

While our automated testing focuses on randomized average-case analysis, manual tests with specific edge cases (e.g., sorted sequences, sequences with all identical numbers) were also conducted during development to ensure correctness.

3.3 Experimental Results

The performance of the four algorithms was measured according to the workflow. The collected data, representing the average runtime for various input sizes, is summarized in Table 1. The data clearly shows the vastly different scales of input that each algorithm can handle within a reasonable time frame.

Table 1: Average Runtime (ms) of Algorithms for Different Input Sizes (N)

Alg. 1: Optimized DP ($O(N \log V)$)		**Alg. 2: Direct DP** ($O(N^2)$)		**Alg. 3: Complementary DP** ($O(N \cdot V)$)		**Alg. 4: Brute Force** ($O(2^N)$)	
N	**Runtime**	**N**	**Runtime**	**N**	**Runtime**	**N**	**Runtime**
10,000	0.5008	500	1.0838	500	57.8340	10	0.0035
20,000	1.0063	1,000	6.1350	1,000	114.3560	12	0.0144
30,000	1.5405	1,500	16.4296	1,500	169.9530	14	0.0612
40,000	2.0053	2,000	32.0261	2,000	223.8790	16	0.2369
50,000	2.5030	2,500	53.0495	2,500	276.4900	18	0.9734
60,000	3.0081	3,000	79.6642	3,000	338.5850	20	3.9137
70,000	3.4575	3,500	110.7276	3,500	398.6690	22	15.7333
80,000	4.0494	4,000	147.5008	4,000	455.9500	-	-
90,000	4.5015	4,500	187.7353	4,500	503.7890	-	-
100,000	4.8938	5,000	235.5554	5,000	561.1780	-	-

4 Performance Analysis

In this section, we provide a formal complexity analysis of our main algorithm and then analyze the empirical results from our experiments, connecting them to the theoretical complexities of all four algorithms.

4.1 Complexity Analysis of the Optimized Algorithm

Our primary solution, Algorithm 1, employs a complementary counting strategy optimized with a Fenwick Tree.

Time Complexity: $O(N \log V)$ The algorithm's runtime is dominated by the main loop that iterates through each of the N elements of the input sequence. Inside this loop, for each element a_i, we perform the following operations:

1. **Querying for previous counts:** We need to calculate the sum of `dp` values for all v such that $|a_i - v| > m$. This is broken down into two prefix sum queries on the Fenwick Tree: `query(a_i - m - 1)` and `query(MAX_VAL) - query(a_i + m)`. Each query operation on a Fenwick Tree of size V (where V is `MAX_VAL`) takes $O(\log V)$ time.

2. **Updating the count:** After calculating the number of new ugly subsequences ending at a_i, we must update the Fenwick Tree at index a_i with this new count. This `update` operation also takes $O(\log V)$ time.

Since these $O(\log V)$ operations are performed for each of the N elements, the total time complexity is $O(N \log V)$.

Space Complexity: $O(V)$ The space complexity is determined by the storage required for the main data structures.

- The input array requires $O(N)$ space.
- The Fenwick Tree, which is indexed by the values of the elements, requires an array of size V (`MAX_VAL`).

In our problem setting, V is a fixed constant (10^5) that is independent of N. Therefore, the dominant factor is the Fenwick Tree, making the overall space complexity $O(V)$.

4.2 Analysis of Experimental Results

The performance data is visualized in Figure 1, which uses a log-log scale to effectively compare algorithms across vast differences in input size and runtime.

Algorithm 4: Brute Force ($O(2^N)$, Red Curve) As expected, the brute-force algorithm's curve appears on the far left of the plot ($N \in [10, 22]$). It exhibits a distinct, sharp upward curve, rather than a straight line. This is the classic signature of exponential growth on a log-log scale, visually confirming that its complexity is not polynomial.

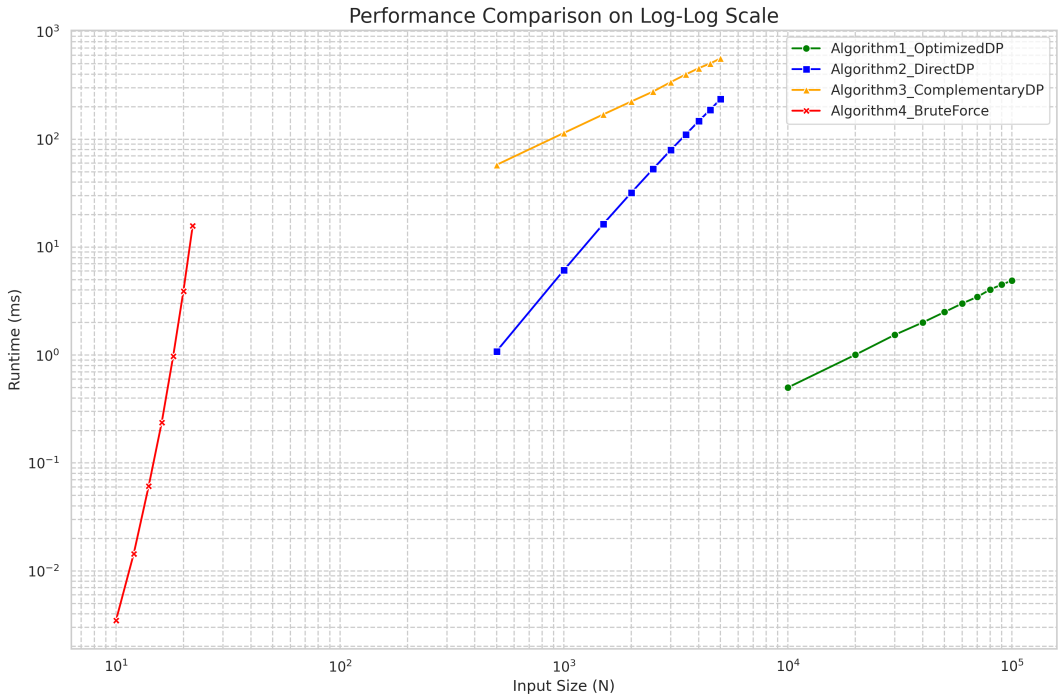

Figure 1: Performance comparison of all four algorithms on a log-log scale. This plot clearly distinguishes the growth rates of algorithms with different time complexities.

Algorithm 2: Direct DP ($O(N^2)$, Blue Curve) This algorithm's data points form a clear straight line. On a log-log plot, a polynomial complexity of $O(N^k)$ corresponds to a line with a slope of k. The observed slope is approximately 2, which perfectly aligns with its theoretical time complexity of $O(N^2)$.

Algorithm 3: Complementary DP without Fenwick Tree ($O(N \cdot V)$, Orange Curve) This algorithm's curve also appears as a straight line. Its time complexity, $O(N \cdot V)$, behaves like $O(N)$ in our tests because V is a large constant. This is reflected in the plot, where its line has a slope of approximately 1.

Algorithm 1: Optimized DP with Fenwick Tree ($O(N \log V)$, Green Curve) This, our final solution, is represented by the green line on the far right. It also appears as a straight line with a slope of approximately 1, consistent with its near-linear complexity ($O(N \log V)$ behaves like $O(N)$ as $\log V$ is constant). The most crucial observation is its low vertical position, indicating extremely low runtimes even for very large N.

4.3 Comparative Analysis and Key Observations

Asymptotic Behavior vs. Constant Factors The most interesting comparison is between Algorithm 2 ($O(N^2)$) and Algorithm 3 ($O(N \cdot V)$). Although Algorithm 2 has a worse asymptotic complexity with respect to N, its curve is positioned vertically lower than Algorithm 3's. This means that for the same input size N, the $O(N^2)$ algorithm was actually faster. This phenomenon is explained by the role of **hidden constant factors** in Big O notation. The runtime of Algorithm 3 is roughly $c_3 \cdot N \cdot V$, while for Algorithm 2 it is $c_2 \cdot N^2$. In our case, V is very large (10^5), making the constant factor $c_3 \cdot V$ enormous.

In contrast, the operations within the inner loop of Algorithm 2 are simple, resulting in a much smaller constant c_2. For the tested range of N, the inequality $c_2 \cdot N^2 < c_3 \cdot N \cdot V$ holds true. However, the steeper slope of the blue line guarantees that, given a large enough N, it will eventually cross and surpass the orange line.

The Power of Optimization Comparing Algorithm 1 ($O(N \log V)$) with Algorithm 3 ($O(N \cdot V)$) starkly illustrates the power of using an appropriate data structure. Both algorithms share the same core DP logic, but the introduction of the Fenwick Tree reduces the complexity of the summation step from $O(V)$ to $O(\log V)$. This optimization effectively reduces the massive constant factor tied to V to a much smaller one tied to $\log V$, resulting in a speedup of several orders of magnitude. The green line's flat trajectory and low position on the graph are a direct testament to this success.

5 Conclusion

In this project, we successfully addressed the problem of counting "beautiful subsequences" by designing, implementing, and analyzing several algorithms. Our primary solution, which employs a complementary counting strategy combined with dynamic programming and a Fenwick Tree for optimization, proved to be highly efficient, achieving a time complexity of $O(N \log V)$.

6 Declaration

We hereby declare that all the work done in this project is of our own independent effort.